# Department of Software Engineering


## Software Verification and Validation Lab

## Semester Project Topic: Hospital Management System

## Project Report

---

| Submitted by | |
|---|---|
| **Members** | **Laiba Majeed – Fa2023-BSSE-125** |
| | **Tooba Butt – Fa2023-BSSE-179** |
| | **Ahmad Saleemi – Fa2023-BSSE-189** |
| **Semester** | **6th Semester** |
| **Section** | **D Section** |
| **Domain Title:** | **Hospital Management System** |

| Submitted To |
|---|
| **Maam Jawaria Ramzan** |

# Table of Contents

# Table of Figures

# List of Tables

# CHAPTER NO 1: INTRODUCTION

## 1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the functional and non-functional requirements of the Hospital Management System (HMS). It provides a structured and formal description of the system to ensure a clear understanding among developers, testers, and evaluators. This document also serves as the foundation for formal modeling, verification, and validation using Z Notation, VDM-SL, and Alloy.

## 1.2 Scope

The Hospital Management System is a secure, role-based software solution designed to manage hospital operations efficiently. The system supports Admin, Doctor, Nurse, and Receptionist roles and includes functionalities such as patient registration, admission and discharge management, room allocation, doctor assignment, billing, emergency handling, medical record maintenance, and audit logging. The system ensures data consistency, operational accuracy, and secure healthcare management.

## 1.3 Problem Statement

Traditional hospital management processes often rely on manual or disconnected systems that lead to operational inefficiencies, data inconsistency, delayed responses, and patient safety risks. Common problems include incorrect patient tracking, room allocation conflicts, billing errors, and poor coordination among hospital departments. The proposed Hospital Management System aims to provide a centralized and formally verified solution that ensures reliable, secure, and consistent hospital operations.

## 1.4 Objectives

The primary objectives of this project are:

- To define and document complete functional and non-functional requirements for the HMS.
- To develop formal models using Z Notation, VDM-SL, and Alloy.
- To ensure correctness, consistency, and validation of hospital operations through formal verification techniques.
- To maintain secure, role-based access control and reliable patient management.
- To establish measurable validation criteria and system traceability using Git and GitHub.

# CHAPTER NO 2: SOFTWARE REQUIREMENTS

## 2.1. Functional Requirements

*Table 1: Functional Requirements*

| Req ID | Requirement |
|---|---|
| **FR1** | The system shall support secure user authentication with role-based access control for Admin, Doctor, Nurse, and Receptionist. |
| **FR2** | The system shall allow registration, update, and management of patient records. |
| **FR3** | The system shall assign doctors to registered patients based on defined hospital rules. |
| **FR4** | The system shall allocate rooms to patients with a constraint of one patient per room. |
| **FR5** | The system shall manage patient admission process in the system. |
| **FR6** | The system shall manage patient discharge process and automatically free the assigned room. |
| **FR7** | The system shall support assignment of doctors to patients based on system rules. |
| **FR8** | The system shall generate billing records based on patient services and hospital stay. |
| **FR9** | The system shall maintain complete medical history and treatment records for each patient. |
| **FR10** | The system shall handle emergency cases, including prioritization of critical patients for immediate attention. |

## 2.2. Non-Functional Requirements

*Table 2: Non Functional Requirements*

| Req ID | Requirement |
|---|---|
| **NFR1** | The system shall enforce secure authentication and role-based access control. |
| **NFR2** | The system shall ensure data consistency across patient, doctor, and room entities. |
| **NFR3** | The system shall ensure high availability during emergency operations. |
| **NFR4** | The system shall maintain audit logs of all critical actions. |
| **NFR5** | The system shall ensure acceptable response time for all operations. |

## 2.3. System Users

- Admin: Full control over system configuration, users, and hospital data.
- Doctor: Responsible for diagnosis, treatment, and updating patient medical records.
- Nurse: Responsible for patient care and monitoring assigned patients.
- Receptionist: Responsible for patient registration, admission, and administrative tasks.

# CHAPTER NO 3: METHODOLOGY

This project follows a structured formal methods lifecycle composed of five major phases:

## 3.1. Requirement Engineering

The first phase involved producing a complete Software Requirements Specification (SRS). Functional requirements were identified covering authentication, patient registration, doctor assignment, room allocation, billing, medical records, and emergency handling. Non-functional requirements were also specified including security, availability, data consistency, audit logging, and performance.

A Requirement Defect Taxonomy Table was produced identifying defects of ambiguity, inconsistency, and non-verifiability in five requirements: patient room assignment, critical patient handling, doctor availability, billing generation, and emergency alerting. GitHub Issues were created to track resolution of each defect.

## 3.2. Formal Modeling – Z Notation

The HMS state schema was formally specified using Z Notation in the CZT tool. The main state schema HMS defined the system state including sets of patients, admitted patients, doctors, rooms, and available rooms, along with relations for room assignment, patient status, and doctor assignment.

Six key invariants were specified: admitted is a subset of patients; assignedRoom is a relation between admitted patients and rooms; each room holds at most one patient (capacity constraint); critical patients must be in admitted; discharged patients must not be in admitted; and all assignments apply only to admitted patients.

Operations specified include AdmitPatient, AssignRoom, DischargePatient, AssignDoctor, MarkCritical, ReleaseRoom, and TransferPatient. Each operation defines its preconditions and postconditions using Z schema notation., TransferPatient) are each modeled as delta schemas specifying pre-conditions and state transitions.

## 3.3. Functional Specification — VDM

The VDM-SL specification modeled HMS operations as pure functions. Types were defined using token types for PATIENT, ROOM, and DOCTOR, with PatientStatus as a union type. Each operation was specified with its type signature, precondition (pre), and postcondition (post).

Functions specified include addPatient, allocateRoom, discharge, assignDoctor, markCritical, getAvailableRooms, and transferPatient. Each function is total over its valid inputs and partial otherwise, with preconditions enforcing valid state before execution.

## 3.4. Strcutural Verification — Alloy

The Alloy Analyzer was used to model the static structure of the HMS and verify key assertions. Signatures (sig) model the core entities. Facts (fact) encode structural invariants such as OnePatientPerRoom and ValidRoomAssign. Assertions (assert) are bounded-model-checked by the Alloy Analyzer for scopes of 1 Hospital, 3 Patients, 3 Rooms, and 3 Doctors. The Analyzer exhaustively explores all configurations within the scope to confirm no counterexample exists.

## 3.5. Validation and Security

Validation criteria were derived directly from the SRS. Measurable criteria were defined for authentication, performance (response time under 2 seconds), data consistency, room allocation correctness, emergency handling (under 1 minute), billing accuracy, audit logging (100% of critical actions), and system availability.

A CI pipeline was implemented using GitHub Actions triggered on push and pull request events to the main branch. The pipeline includes build, test, security, and reporting stages ensuring system integrity is automatically validated on every code change.

# CHAPTER NO 4: TOOLS USED

*Table 3: Tools to be used*

| Tool | Purpose | Phase |
|---|---|---|
| **Z Notation** | Formal specification of system state and operations | Formal Modeling |
| **VDM-SL (Overture)** | Functional specification with pre/post conditions | Functional Spec |
| **Alloy Analyzer** | Structural verification and assertion checking | Verification |
| **Git** | Version control, branching, commit history | All Phases |
| **GitHub** | Issue tracking, collaboration, requirement traceability | All Phases |
| **Word** | Documentation and report generation | All Phases |

# CHAPTER NO 5: RESULT

## 5.1. Software Requirements Specification

The SRS produced eight functional requirements and five non-functional requirements covering authentication, patient management, room allocation, doctor assignment, billing, medical records, emergency handling, and audit logging. The system was designed for four roles: Admin, Doctor, Nurse, and Receptionist.

## 5.2. Requirement Defect Taxonomy

Five requirements were analyzed for defects. The taxonomy revealed systematic issues across all analyzed requirements:

*Table 4: Requirement Defect Taxonomy*

| Requirement | Ambiguity | Inconsistency | Non-verifiability |
|---|---|---|---|
| Patient room assignment | Assignment not time/state-defined | Conflicts from multiple states | No time/state constraints |
| Critical patient handling | "Immediate" not time-defined | Conflicts with room constraints | No measurable time condition |
| Doctor availability | "Availability" not defined | Scheduling overlaps undefined | No measurable criteria |
| Billing generation | Billing rules unspecified | Discount/service rule conflicts | No formula defined |
| Emergency alert system | Trigger conditions undefined | Duplicate alerts possible | No validation criteria |

## 5.3. Github Issue Tracking Log

*Table 5: Github Issue Tracking Log*

| Issue ID | Title | Type | Status |
|---|---|---|---|
| #1 | Define patient-room allocation rules | Requirement | Open |
| #2 | Resolve critical patient handling logic | Bug | Open |

| | | | |
|---|---|---|---|
| #3 | Implement authentication system | Feature | In Progress |
| #4 | Define billing calculation logic | Requirement | Open |
| #5 | Enforce room capacity constraint | Feature | Open |
| #6 | Implement audit logging | Feature | Open |

## 5.4. Z Notation Model

The HMS Z schema was successfully formalized with the following state components and invariants:

- State: patients, admitted, doctors, rooms, availableRooms, assignedRoom, patientStatus, doctorAssignment
- Key invariant: ∀ r : ROOM • card { p : PATIENT | p ↦ r ∈ assignedRoom } ≤ 1 (one patient per room)
- Critical patient invariant: ∀ p : PATIENT • patientStatus(p) = critical ⇒ p ∈ admitted
- Discharge invariant: ∀ p : PATIENT • patientStatus(p) = discharged ⇒ p ∉ admitted
- Room availability: availableRooms = rooms \ ran assignedRoom

Seven operations were specified: AdmitPatient, AssignRoom, DischargePatient, AssignDoctor, MarkCritical, ReleaseRoom, and TransferPatient. Each operation uses delta schemas (ΔHMS) to define pre-conditions and the effect on system state.

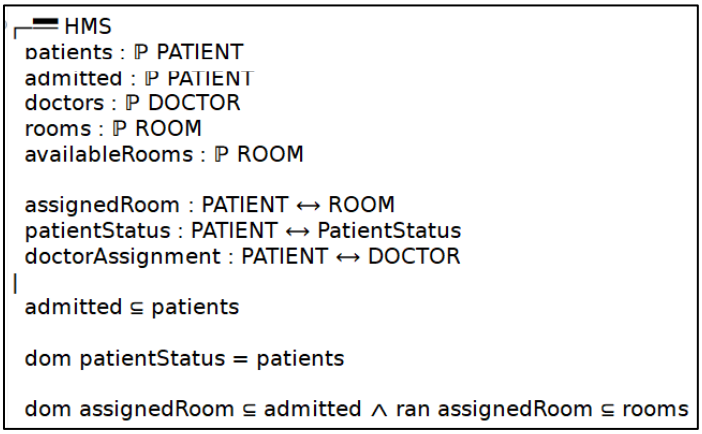


*Figure 1: State Schema*

## Operation in CZT:

```
-- ============================
-- OPERATION 1: AdmitPatient
-- ============================
AdmitPatient [OP]
ΔHMS
p? : PATIENT

-- patient must not already exist
p? ∉ patients

patients' = patients ∪ {p?}
admitted' = admitted ∪ {p?}
patientStatus' = patientStatus ⊕ {p? ↦ normal}
```

*Figure 2: Operation - AdmitPatient*

## Result in CZT:

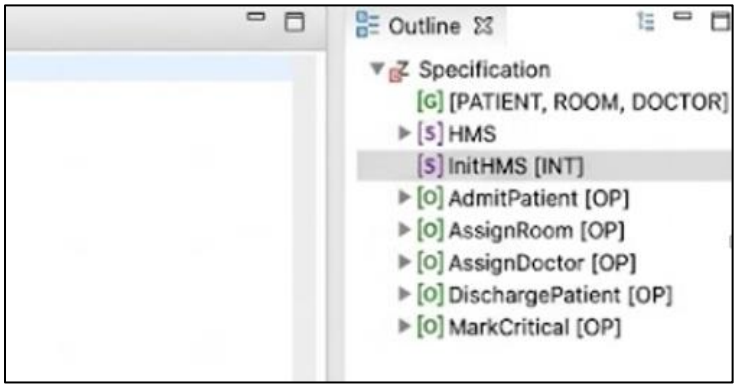


*Figure 3: CZT Reuslt in Outlines*

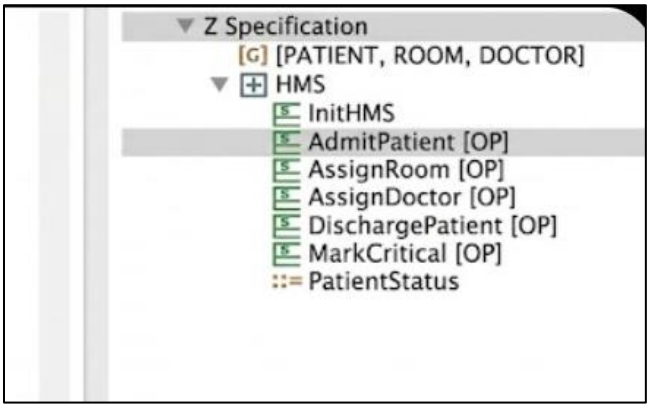


*Figure 4: CZT Result 2*

## 5.5. VDM-SL Fucntional Specification

The VDM specification encodes seven operations as typed functions with explicit pre- and post-conditions. Critical design decisions include:

- PATIENT, ROOM, and DOCTOR are token types ensuring opaque identity.
- Map operations (munion) handle partial updates to doctor and room assignments.
- Pre-conditions guard against invalid state transitions (e.g., assigning a room already occupied).
- Post-conditions mathematically certify the result of each operation.

**Vdm Function:**

```
PatientStatus = <normal> | <critical> | <discharged>;

------------------------------------------------------------
-- 1. addPatient
------------------------------------------------------------

addPatient: set of PATIENT * PATIENT -> set of PATIENT
addPatient(patients, p) ==
    patients union {p}

pre p not in set patients
post RESULT = patients union {p};
```

*Figure 5: Vdm Function*

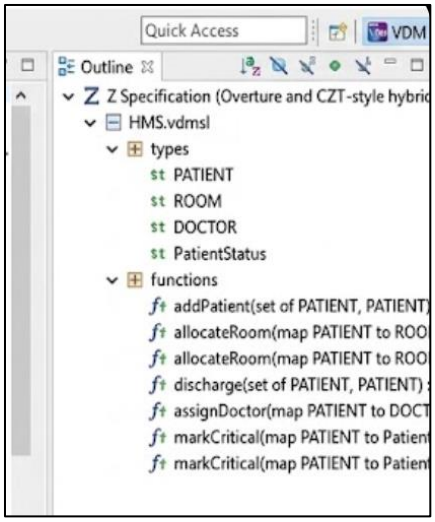


*Figure 6: Outline in VDM*

## 5.6. Alloy Structure Verification

The Alloy model defines three signatures (Patient, Doctor, Room) and a Hospital signature with relations to each. Five facts enforce structural invariants. Four assertions were checked:

**Alloy Sig:**

```
sig Patient {}
sig Doctor {}
sig Room {}

sig Hospital {
   patients: set Patient,
   doctors: set Doctor,
   rooms: set Room,
   assigned: Patient -> lone Room,
   doctorOf: Patient -> lone Doctor
}
```

*Figure 7: Alloy Sigs + Relations*

**Alloy fact and asert:**

```
assert NoDoubleBooking {
   all h: Hospital, r: Room |
      lone h.assigned.r
}

assert SingleRoomPerPatient {
   all h: Hospital, p: Patient |
      lone p.(h.assigned)
}
```

*Figure 8: Alloy fact + assert*

## Alloy Results:

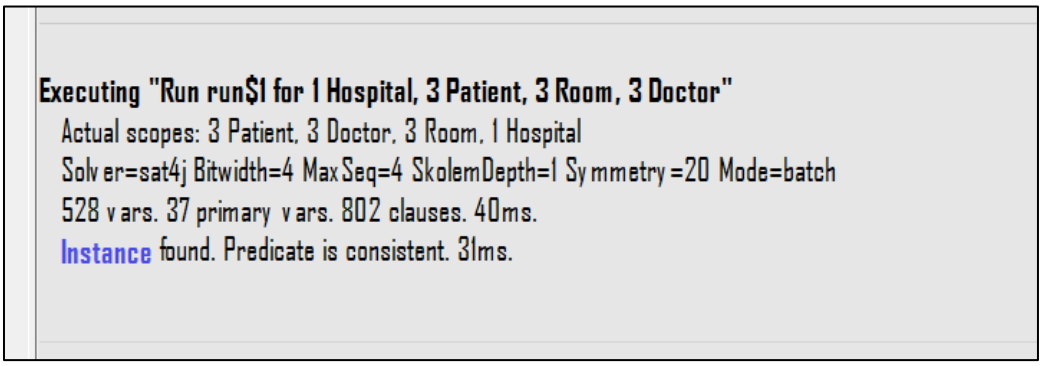


*Figure 9: Alloy Result*

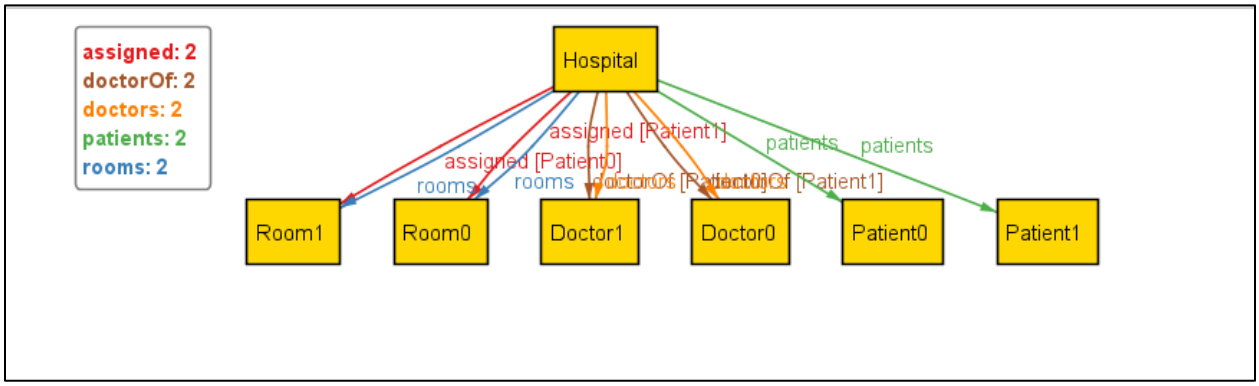


*Figure 10: Alloy Result -Figure*

## Counter Example:

The assertion **WrongAssumption** is intentionally designed to be false in order to demonstrate counterexample analysis in Alloy. It assumes that every patient must always have an assigned room in the hospital system. However, this is not guaranteed by the model's constraints, since a patient can exist in the hospital without being assigned a room. When the Alloy Analyzer executes the check command for this assertion, it generates a counterexample showing at least one patient who belongs to the hospital but has no room assignment. This clearly violates the assertion and proves that the assumption is incorrect. This counterexample helps validate the model by highlighting missing or invalid assumptions and demonstrates how Alloy is used to detect logical inconsistencies in system specifications.

## 5.7. Measurable Validation Criteria

*Table 6 : Measurable Validation Criteria*

| Expectation | Measurable Criteria |
|---|---|
| Secure authentication | Login success only for valid credentials with role-based access control enabled |
| System performance | Response time ≤ 2 seconds for patient, doctor, and room operations |
| Data consistency | Patient–doctor–room relationships remain consistent with no orphan or duplicate assignments |
| Room allocation correctness | Each room holds maximum 1 patient at any time (1:1 constraint enforced) |
| Emergency handling | Critical patient marked and processed within ≤ 1 minute of detection |
| Billing accuracy | Billing records generated with 100% consistency based on patient stay and services |
| Audit logging | 100% of critical actions logged with timestamp, user role, and operation type |
| System availability | Core system remains operational during concurrent user access without data loss |
| Access control | Unauthorized users cannot access restricted modules |
| CI validation | All automated tests pass successfully in GitHub Actions pipeline |

## 5.8. Validation Checklist

*Table 7: Validation Checklist*

| ID | Requirement | Validation Criteria | Method | Status |
|---|---|---|---|---|
| 1 | User Login | Role-based authentication works correctly and rejects invalid access | Testing | Pending |
| 2 | Patient Registration | Patient data is stored without duplication and with required fields | Testing | Pending |

| | | | | |
|---|---|---|---|---|
| 3 | Doctor Assignment | Only registered doctors can be assigned to valid patients | Testing | Pending |
| 4 | Room Allocation | One patient per room constraint is strictly enforced | Testing | Pending |
| 5 | Admission/Discharge | Patient state changes correctly update room availability | Testing | Pending |
| 6 | Billing System | Billing is generated accurately based on defined rules | Testing | Pending |
| 7 | Emergency Handling | Critical patients are prioritized in allocation and logged | Inspection | Pending |
| 8 | Audit Logging | All critical actions are recorded with full traceability | Inspection | Pending |
| 9 | Data Consistency | No inconsistent or orphan records exist in system relations | Testing | Pending |
| 10 | System Security | Unauthorized access attempts are blocked successfully | Security Testing | Pending |
| 11 | Access Control | Users can access only permitted modules based on role | Inspection | Pending |
| 12 | CI Pipeline Validation | GitHub Actions workflow executes successfully without build/test failure | Automated Testing | Pending |

# CHAPTER NO 6: CONCLUSION

This project successfully applied formal methods to the design, specification, and verification of a Hospital Management System (HMS). During the Requirements Engineering phase, all functional and non-functional requirements were documented in the Software Requirements Specification (SRS). Requirement analysis and defect identification helped detect ambiguities, inconsistencies, and missing validations, ensuring a clearer and more reliable system specification.

The Z Notation model formally defined the HMS state and operations using mathematical specifications. Important system constraints such as one patient per room, valid patient admission and discharge, and proper doctor assignment were verified through formal invariants. The VDM-SL specification further modeled system operations using typed functions with clear pre-conditions and post-conditions, improving precision and consistency between specification and implementation.

The Alloy Analyzer was used for bounded model checking to verify important structural properties of the system. Assertions related to room allocation, patient membership, and doctor assignments were successfully validated without counterexamples. Overall, the project demonstrated how formal methods, combined with tools like Git and GitHub, can improve software reliability, correctness, traceability, and system validation in modern software engineering practices.

## GITHUB LINK:

https://github.com/laibah-majeed/HMS-SVV-Project